

# Gherkin + Java + Selenium: Sıfırdan HERO Seviyesine 10 Dakikalık Hızlandırılmış Eğitim

Sen zaten Selenium/Playwright ve Java biliyorsun. Burada sadece **Gherkin syntax**'ı ve bunu **Cucumber-JVM** üzerinden Selenium'a nasıl bağlayacağını öğreneceksin. Kod bloklarını kopyala-çalıştır mantığıyla ilerle.

## 1) Gherkin Nedir? (30 saniye)

Gherkin, test senaryolarını **doğal dilde**, **yapılandırılmış** biçimde yazmanı sağlayan bir DSL (Domain Specific Language). Amaç: iş analisti / PO / QA / developer aynı **.feature** dosyasını okuyup anlayabilsin (BDD — Behavior Driven Development). Gherkin dosyasının kendisi hiçbir şey çalıştırmaz; her satırı Java'da bir **step definition** metoduna eşlersin. Bu eşleme işini **Cucumber-JVM** yapar, Selenium ise tarayıcı otomasyonunu üstlenir ([Cucumber resmi dokümantasyon](#)).

Zincir şu şekilde işler:

.feature dosyası (Gherkin) → Cucumber step definitions (Java) → Selenium WebDriver

## 2) Temel Anahtar Kelimeler

Her **.feature** dosyası şu iskelete sahiptir:

```
Feature: Kullanıcı girişi
  Kullanıcıların sisteme giriş yapabilmesi gerekiyor

  Scenario: Geçerli kullanıcı adı ve şifre ile başarılı giriş
    Given kullanıcı login sayfasındadır
    When kullanıcı "isa" kullanıcı adını girer
    And kullanıcı "1234" şifresini girer
    And giriş butonuna tıklar
    Then ana sayfaya yönlendirilir
    And hoş geldin mesajı görünür
```

| Anahtar Kelime | Anlamı                                 | Not                                                   |
|----------------|----------------------------------------|-------------------------------------------------------|
| Feature        | Test edilen özelliğin başlığı          | Dosya başına 1 tane, açıklama satırları serbest metin |
| Scenario       | Tek bir test case (bir "it should...") | Bir feature'da birden çok olabilir                    |
| Given          | Ön koşul / başlangıç durumu            | "Sistem şu durumdayken..."                            |
| When           | Eylem / tetikleyici                    | "Kullanıcı şunu yaptığında..."                        |

| Anahtar Kelime | Anlamı                     | Not                                                                 |
|----------------|----------------------------|---------------------------------------------------------------------|
| Then           | Beklenen sonuç / assertion | "Şu gerçekleşmeli"                                                  |
| And / But      | Önceki adımın devamı       | Okunabilirlik için; Cucumber arkada Given/When/Then olarak ele alır |

**HERO NOTU:** Given/When/Then Cucumber için teknik olarak birbirinden farksızdır — hepsi aynı step definition havuzuna eşleşir. Ayrım tamamen okunabilirlik içindir (Given-When-Then = Arrange-Act-Assert).

### 3) Background — Tekrarlanan Given'ları Tekilleştir

Bir feature içindeki her senaryo aynı ön koşulla başlıyorsa:

Feature: Sepet işlemleri

Background:

Given kullanıcı giriş yapmıştır  
And ürünler sayfasındadır

Scenario: Ürünü sepete ekleme

When "Laptop" ürününü sepete ekler  
Then sepette 1 ürün görünür

Scenario: Ürünü sepetten çıkarma

Given sepette "Laptop" ürünü vardır  
When ürünü sepetten çıkarır  
Then sepet boş görünür

Background her Scenario'dan önce otomatik çalışır — Java'daki @BeforeEach gibi düşün.

### 4) Scenario Outline + Examples — Data-Driven Test

Aynı senaryoyu farklı verilerle N kez çalıştırmak istediğinde (parametrize test):

Scenario Outline: Farklı kullanıcılarla giriş denemesi

Given kullanıcı login sayfasındadır  
When kullanıcı "<username>" ve "<password>" ile giriş yapar  
Then sonuç mesajı "<mesaj>" olur

Examples:

|                                    |  |
|------------------------------------|--|
| username   password   mesaj        |  |
| isa   1234   Giriş başarılı        |  |
| isa   yanlış   Şifre hatalı        |  |
| boş   1234   Kullanıcı adı gerekli |  |

Cucumber, Examples tablosundaki her satır için senaryoyu bir kez çalıştırır ve <parametre> yer tutucularını değiştirir. JUnit'teki @ParameterizedTest + @CsvSource'un Gherkin karşılığı budur.

## 5) Data Tables — Step'e Tablo Parametresi Geçirme

```
Scenario: Birden fazla ürünü sepete ekleme
  When aşağıdaki ürünleri sepete ekler:
    | ürün   | adet |
    | Laptop | 1    |
    | Mouse  | 2    |
  Then sepete 2 farklı ürün olur
```

Java tarafında `io.cucumber.datatable.DataTable` olarak yakalanır (aşağıda örnek var).

## 6) Doc Strings — Çok Satırlı Metin

```
Given aşağıdaki JSON payload gönderilir:
"""
{
  "username": "isa",
  "password": "1234"
}
"""
```

Step metodunda `String` parametresi olarak gelir.

## 7) Tags — Senaryoları Filtreleme

```
@smoke @login
Scenario: Başarılı giriş
...

@regression @slow
Scenario: Şifre sıfırlama akışı
...
```

Sadece `@smoke` etiketlilerini çalıştırmak için Maven/Runner'a `cucumber.filter.tags=@smoke` parametresi verilir. CI pipeline'larında `smoke/regression` ayrımı için kritik.

## 8) Proje Kurulumu (Maven, güncel — 2026 itibarıyla)

```
<properties>
  <cucumber.version>7.34.7</cucumber.version>
  <selenium.version>4.27.0</selenium.version>
  <junit.version>5.11.0</junit.version>
</properties>

<dependencies>
  <!-- Cucumber core + step definition API -->
  <dependency>
    <groupId>io.cucumber</groupId>
    <artifactId>cucumber-java</artifactId>
```

```

        <version>${cucumber.version}</version>
        <scope>test</scope>
    </dependency>

    <!-- Cucumber'ı JUnit5 Platform üzerinden çalıştırmak için -->
    <dependency>
        <groupId>io.cucumber</groupId>
        <artifactId>cucumber-junit-platform-engine</artifactId>
        <version>${cucumber.version}</version>
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>org.junit.platform</groupId>
        <artifactId>junit-platform-suite</artifactId>
        <version>1.11.0</version>
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>${junit.version}</version>
        <scope>test</scope>
    </dependency>

    <!-- Selenium -->
    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>${selenium.version}</version>
        <scope>test</scope>
    </dependency>
</dependencies>

```

**Kaynak:** [cucumber-java Maven Central](#), [Cucumber-JVM GitHub sürümleri](#). Not: JUnit4 dönemindeki `@RunWith(Cucumber.class)` artık **legacy** — Cucumber 7'den beri JUnit5 Platform Suite (`@Suite + @IncludeEngines("cucumber")`) standart yaklaşımdır.

## Klasör Yapısı (standart konvansiyon)

```

src/test/java/com/isa/steps/LoginSteps.java
src/test/java/com/isa/hooks/Hooks.java
src/test/java/com/isa/runner/CucumberTestSuite.java
src/test/resources/features/login.feature

```

## 9) Runner Sınıfı (JUnit5 Platform Suite)

```
package com.isa.runner;

import org.junit.platform.suite.api.ConfigurationParameter;
import org.junit.platform.suite.api.IncludeEngines;
import org.junit.platform.suite.api.SelectClasspathResource;
import org.junit.platform.suite.api.Suite;

import static io.cucumber.junit.platform.engine.Constants.GLUE_PROPERTY_NAME;
import static io.cucumber.junit.platform.engine.Constants.PLUGIN_PROPERTY_NAME;

@Suite
@IncludeEngines("cucumber")
@SelectClasspathResource("features")
@ConfigurationParameter(key = GLUE_PROPERTY_NAME, value = "com.isa.steps,com.isa.hool")
@ConfigurationParameter(key = PLUGIN_PROPERTY_NAME,
    value = "pretty, html:target/cucumber-report.html")
public class CucumberTestSuite {
}
```

mvn test çalıştırdığında bu sınıf otomatik keşfedilip Cucumber engine'i tetikler ([Cucumber-JVM release notes v7.0.0](#)).

## 10) Hooks — WebDriver Yaşam Döngüsü

```
package com.isa.hooks;

import io.cucumber.java.After;
import io.cucumber.java.Before;
import io.cucumber.java.Scenario;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class Hooks {

    public static WebDriver driver;

    @Before
    public void setUp() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
    }

    @After
    public void tearDown(Scenario scenario) {
        if (scenario.isFailed() && driver != null) {
            byte[] screenshot = ((TakesScreenshot) driver).getScreenshotAs(OutputType.BYTES);
            scenario.attach(screenshot, "image/png", scenario.getName());
        }
        if (driver != null) {
            driver.quit();
        }
    }
}
```

```
        driver.quit();
    }
}
}
```

@Before/@After her Scenario'dan önce/sonra çalışır — TestNG/JUnit'teki

@BeforeMethod/@AfterMethod karşılığı. Başarısız senaryoda otomatik ekran görüntüsü almak HERO seviyesi bir pratiktir.

## 11) Step Definitions — Selenium ile Bağlama

src/test/resources/features/login.feature:

Feature: Kullanıcı girişi

Scenario: Geçerli bilgilerle giriş

Given kullanıcı login sayfasındadır

When "isa" kullanıcı adını ve "1234" şifresini girer

And giriş butonuna tıklar

Then ana sayfaya yönlendirilir

src/test/java/com/isa/steps/LoginSteps.java:

```
package com.isa.steps;

import io.cucumber.java.en.Given;
import io.cucumber.java.en.Then;
import io.cucumber.java.en.When;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

import java.time.Duration;

import static com.isa.hooks.Hooks.driver;
import static org.junit.jupiter.api.Assertions.assertTrue;

public class LoginSteps {

    private WebDriverWait wait() {
        return new WebDriverWait(driver, Duration.ofSeconds(10));
    }

    @Given("kullanıcı login sayfasındadır")
    public void kullanıcı_login_sayfasındadır() {
        driver.get("https://ornek-site.com/login");
    }

    @When("{string} kullanıcı adını ve {string} şifresini girer")
    public void kullanıcı_bilgilerini_girer(String username, String password) {
        driver.findElement(By.id("username")).sendKeys(username);
    }
}
```

```

        driver.findElement(By.id("password")).sendKeys(password);
    }

    @When("giriş butonuna tıklar")
    public void giris_butonuna_tiklar() {
        driver.findElement(By.id("loginBtn")).click();
    }

    @Then("ana sayfaya yönlendirilir")
    public void ana_sayfaya_yonlendirilir() {
        wait().until(ExpectedConditions.urlContains("/dashboard"));
        assertTrue(driver.getCurrentUrl().contains("/dashboard"));
    }
}

```

**Kritik nokta – Cucumber Expressions:** {string}, {int}, {word} gibi yer tutucular Gherkin adımındaki değeri otomatik metod parametresine bağlar; regex de kullanılabilir (^kullanıcı "([^\"]\*)" girer\$) ama Cucumber Expressions daha okunabilir ve varsayılan tercih.

### Data Table Örneği (Java tarafı)

```

@When("aşağıdaki ürünleri sepete ekler:")
public void urunleri_sepete_ekler(DataTable dataTable) {
    List<Map<String, String>> urunler = dataTable.asMaps();
    for (Map<String, String> urun : urunler) {
        // urun.get("ürün"), urun.get("adet")
        addToCart(urun.get("ürün"), Integer.parseInt(urun.get("adet")));
    }
}

```

## 12) Page Object Model ile Birleştirme (HERO Pratiği)

Step definition'lara Selenium locator'larını doğrudan gömmek anti-pattern'dir. Doğru yapı:

```

public class LoginPage {
    private final WebDriver driver;
    private final By usernameField = By.id("username");
    private final By passwordField = By.id("password");
    private final By loginButton = By.id("loginBtn");

    public LoginPage(WebDriver driver) { this.driver = driver; }

    public void login(String user, String pass) {
        driver.findElement(usernameField).sendKeys(user);
        driver.findElement(passwordField).sendKeys(pass);
        driver.findElement(loginButton).click();
    }
}

```

Step sınıfı sadece `new LoginPage(driver).login(username, password);` çağırır. Böylece UI değişikliği tek bir dosyayı etkiler, step definition'lar sabit kalır.

### 13) Çalıştırma

```
mvn test                                # tüm senaryolar
mvn test -Dcucumber.filter.tags="@smoke" # sadece @smoke etiketliler
```

### 14) HERO Seviyesi Checklist — Sık Yapılan Hatalar

- ✗ Step definition içine locator gömmek → ✓ Page Object kullan
- ✗ Bir step'i tek bir feature'a özel yazmak → ✓ Step'leri genel/yeniden kullanılabilir tut ({string} parametrik yap)
- ✗ Scenario içinde implementasyon detayı yazmak ("id=loginBtn'e tıkla") → ✓ İş dilinde yaz ("giriş butonuna tıklar"), teknik detay Java'da kalsın
- ✗ Her senaryoda yeni WebDriver açıp kapatmayı unutmak (memory leak, flaky test) → ✓ @Before/@After hook'larla yönet
- ✗ Dev/test ortam bilgisini step'e hardcode etmek → ✓ System.getProperty / config dosyası / .env kullan
- ✗ Aşırı uzun senaryolar (10+ adım) → ✓ Background ile ortak kısmı ayır, senaryoyu 3-7 adımda tut
- ✗ Paralel çalıştırmada static WebDriver paylaşmak → ✓ ThreadLocal<WebDriver> kullan (paralel Cucumber testlerinde zorunlu)

### Özet — 3 Cümlede Gherkin

1. .feature dosyası iş dilinde ne test edildiğini anlatır (Given/When/Then).
2. Cucumber-JVM, her satırı Java'daki step definition metoduna Cucumber Expressions ile eşler.
3. Step definition içindeki Selenium WebDriver çağrılarını gerçek tarayıcı otomasyonunu yapar; Page Object Model ile bunu locator karmaşasından izole edersin.

Artık sıfırdan gelen biri değil, kurulumu yapıp ilk feature'ını yazabilecek seviyedesin. Sıradaki adım: kendi projende bir smoke senaryo yazıp `mvn test` ile ilk yeşil (veya kırmızı, öğrenmek için) sonucu görmek.